



REPORTE TÉCNICO

Pipeline de observabilidad end-to-end con OpenTelemetry

Docker local · GCP GKE · AWS ECS (placeholder)

Realizado por:

Yerlinson Maturana Serna
Oscar Eduardo Clavijo Alarcón
Camilo Andrés Porras Martín
Edward Daniel Porras Martín

Entregable	Estado
SDK OTel y tres señales	Implementado y probado
Collector local y GCP GKE	Ejecutado con evidencia
Dashboard Grafana	6 paneles con datos
IaC GCP/AWS	Helm y Terraform válidos
Benchmark	Mediciones reales incluidas

Link Github: <https://github.com/YerlinMatu/otel-observability-multicloud>

1. Objetivo y alcance

El sistema captura trazas distribuidas, métricas y logs estructurados de dos microservicios. service-a recibe una orden, valida la entrada y llama por HTTP a service-b. service-b calcula el precio y realiza escritura y lectura en SQLite. El contexto W3C traceparent viaja en la llamada HTTP, por lo que los spans de ambos servicios forman una sola traza.

Flujo de señales

Señal	Origen	Ruta	Uso
Trazas	FastAPI, HTTPX, SQLite y spans custom	OTLP → Collector → Jaeger/X-Ray	Camino de una solicitud
Métricas	SDK OTel y runtime	Endpoint → Prometheus	SLIs y capacidad
Logs	logging JSON + OTLP	Collector → Loki/Cloud Logging/CloudWatch	Eventos con trace_id

Criterios de éxito

Una solicitud debe mostrar spans padre-hijo en service-a y service-b, un span de base de datos y el mismo trace_id en los logs. Prometheus debe marcar todos los targets como UP; el dashboard debe mostrar tráfico, p99, errores, disponibilidad, CPU y errores del Collector.

2. Instrumentación con OpenTelemetry

La aplicación usa FastAPI y el SDK de Python. FastAPIInstrumentor crea spans de servidor; HTTPXClientInstrumentor crea spans de cliente e inyecta Trace Context; el instrumentador SQLite registra operaciones de base de datos mediante cursores instrumentados. Los spans validate-order y calculate-price representan lógica crítica.

Diseño de logs

TraceJsonFormatter serializa timestamp, severidad, mensaje, service.name, deployment.environment, trace_id y span_id. El identificador se toma del span activo y se emite por stdout y OTLP. Esta estrategia permite usar trace_id como pivote desde Grafana Logs hacia Jaeger.

Métricas y SLIs

Panel	Consulta conceptual	Interpretación
Throughput	rate(request_count)	Solicitudes por segundo
Latencia p99	histogram_quantile(0.99, ...)	Cola larga percibida
Errores	5xx / total	Confiabilidad
Disponibilidad	avg(up)	Targets accesibles
CPU	rate(process_cpu_seconds_total)	Costo de cómputo
Collector	send_failed_*	Pérdida de telemetría

Se evita colocar order_id como etiqueta de métrica porque produciría alta cardinalidad; sí es apropiado como atributo de span.

3. Collector y despliegues

Los tres pipelines aplican `memory_limiter` antes de batch para proteger el proceso ante picos. `resource` normaliza atributos de plataforma y ambiente. `batch` reduce llamadas a los backends. OTLP se habilita por gRPC 4317 y HTTP 4318.

Entorno	Trazas	Métricas	Logs
Local	Jaeger por OTLP	Prometheus	Loki
GCP	Jaeger + Cloud Trace	Prometheus + Cloud Monitoring	Cloud Logging
AWS	AWS X-Ray	Prometheus	CloudWatch Logs

GKE

El chart crea namespace, Collector replicado, servicios y deployments. La ejecución real se realizó en un clúster temporal GKE Autopilot llamado `otel-evidence`, región `us-central1`, proyecto `unisabana-kubernetes`. Workload Identity vinculó la ServiceAccount de Kubernetes con una cuenta GCP con permisos mínimos de escritura de trazas, métricas y logs.

ECS Fargate

La tarea agrupa `service-a`, `service-b` y AWS Distro for OpenTelemetry como sidecar; `localhost` simplifica OTLP. La definición incluye roles para X-Ray y CloudWatch y carga la configuración versionada del Collector.

Seguridad

No se codifican credenciales. En cloud se usan identidades de workload, red privada, retención limitada y control de atributos. No deben registrarse tokens, datos personales ni cuerpos completos.

4. Correlación y operación

El procedimiento de diagnóstico parte de un SLI degradado, selecciona una ventana temporal, abre una traza lenta, identifica el span dominante y busca su `trace_id` en logs. Esta navegación conecta el síntoma cuantitativo con la ejecución concreta.

Prueba	Resultado	Evidencia
GET /orders/demo	HTTP 200	Verificado
Jaeger service-a	Traza A → B → SQLite	12 spans; 2 servicios
Logs por trace_id	Mismo ID que Jaeger	Verificado
Prometheus	Todos los targets UP	4/4 UP
Dashboard	Seis paneles	Verificado

5. Evidencia visual de ejecución local

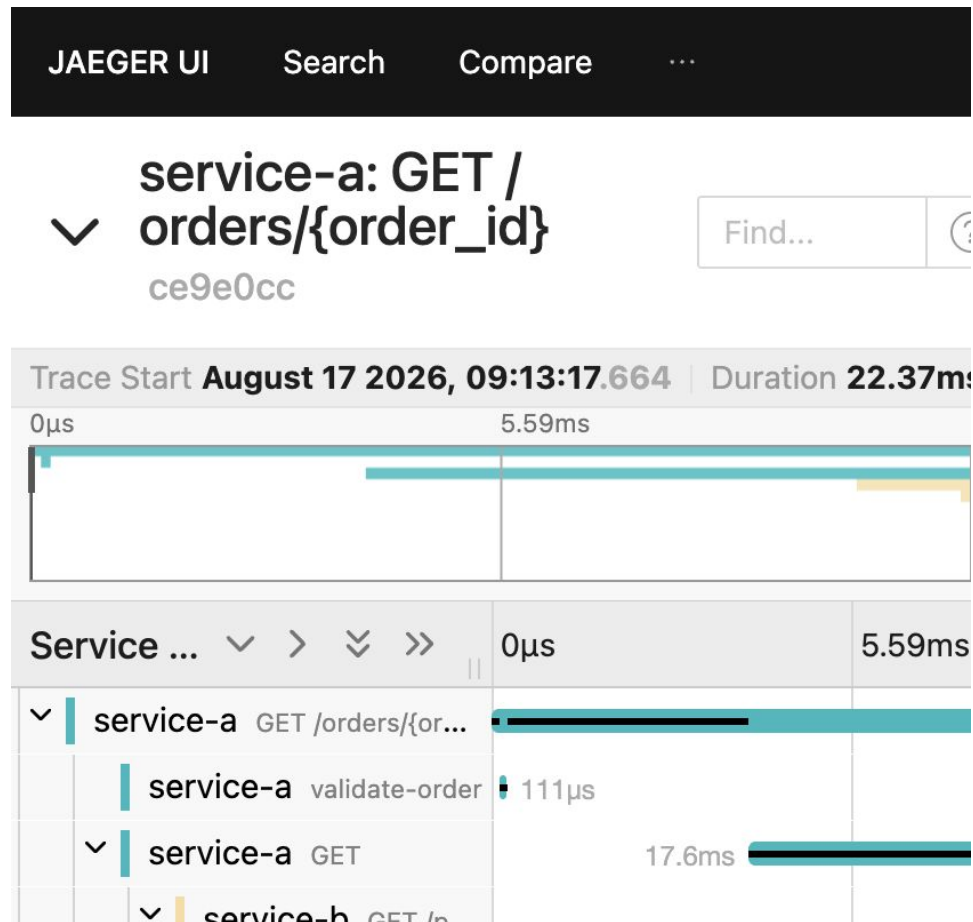


Figura 1. Jaeger: traza distribuida de service-a a service-b.

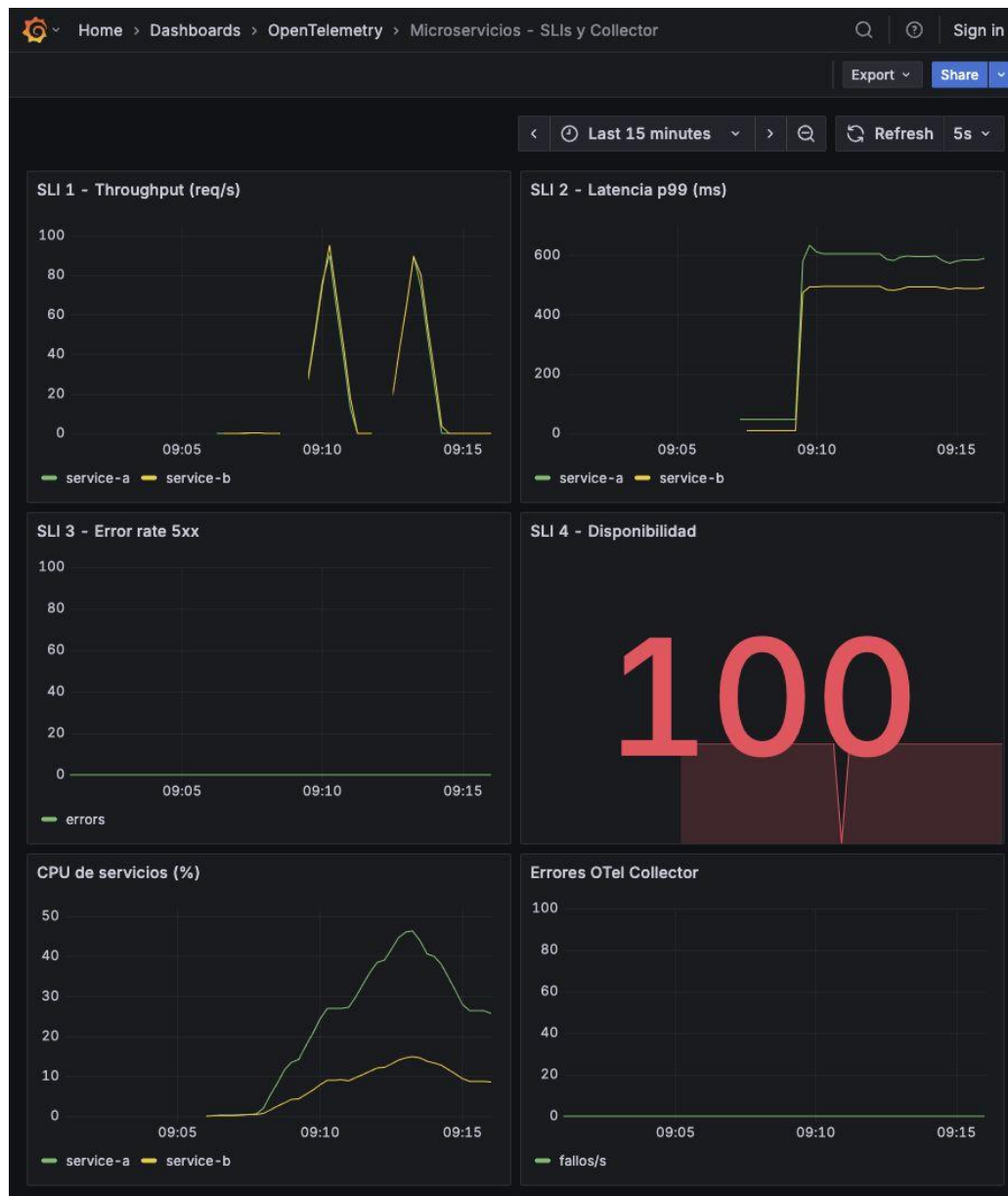


Figura 2. Grafana: dashboard con cuatro SLIs, CPU y errores del Collector.

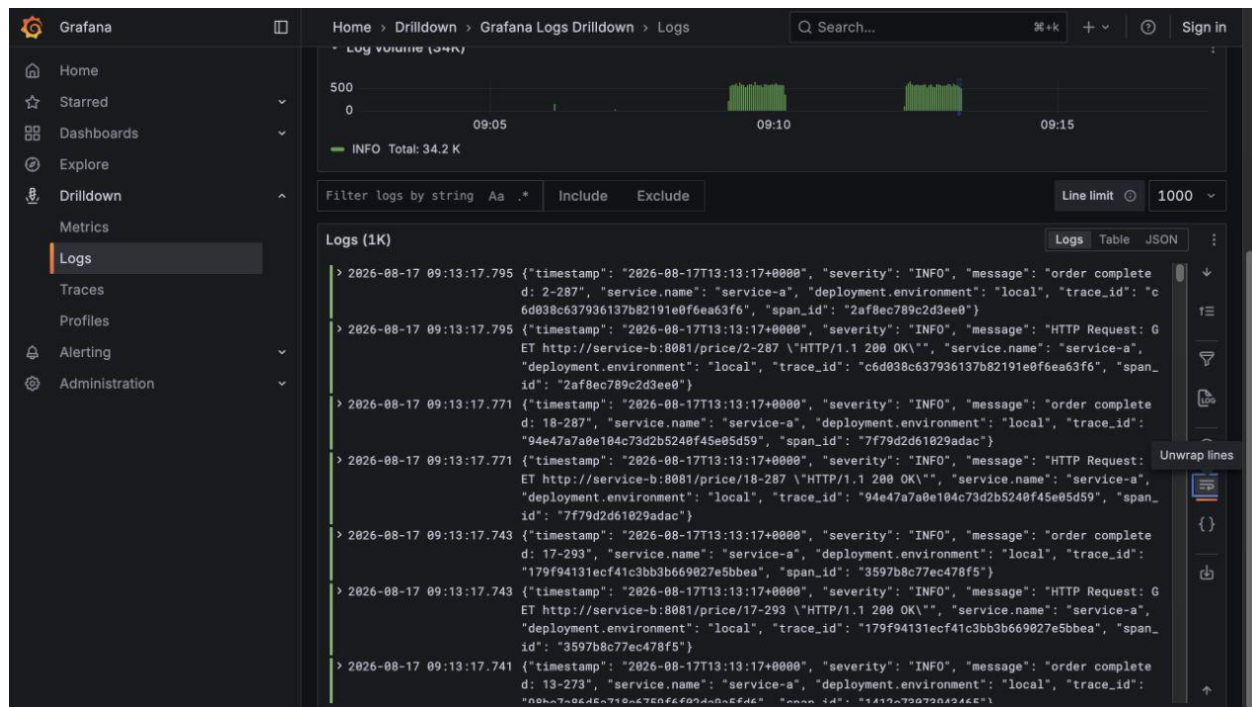


Figura 3. Grafana Logs Drilldown: líneas JSON con trace_id y span_id.

Las evidencias fueron obtenidas del stack Docker local con siete contenedores activos. Jaeger registró trazas de 12 spans con operaciones HTTP, validate-order, calculate-price, CREATE, INSERT y SELECT.

6. Evidencia de ejecución real en GCP GKE

Se desplegaron ocho pods en GKE Autopilot: service-a, service-b, dos réplicas del OTEL Collector, Jaeger, Prometheus, Loki y Grafana. Se generaron 70 solicitudes HTTP exitosas. El ambiente se marcó como gcp-gke y los recursos temporales se eliminaron después de recolectar la evidencia para detener cargos.

Comprobación cloud	Resultado
Pods GKE	8/8 Running; 0 reinicios
Traza distribuida	12 spans; service-a + service-b
Prometheus	4 targets UP
Grafana	6 paneles; disponibilidad 100 %
Cloud Logging	40 eventos OTLP correlacionados con Cloud Trace

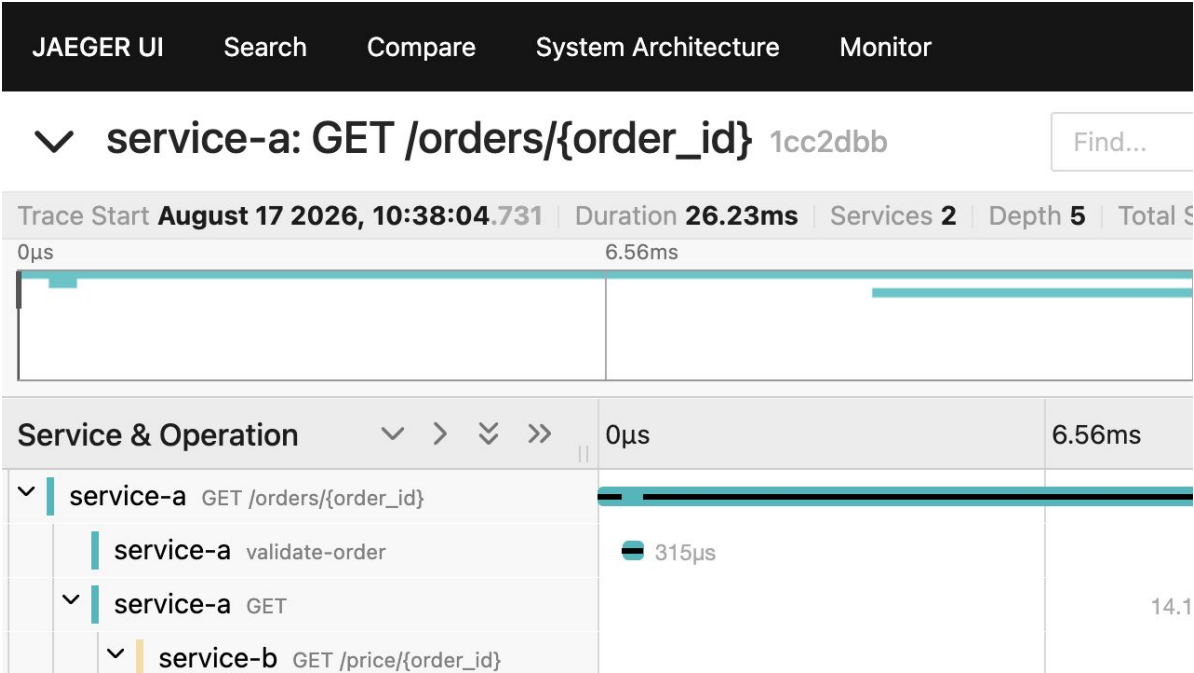


Figura 4. GCP GKE: traza completa en Jaeger con dos servicios y 12 spans.

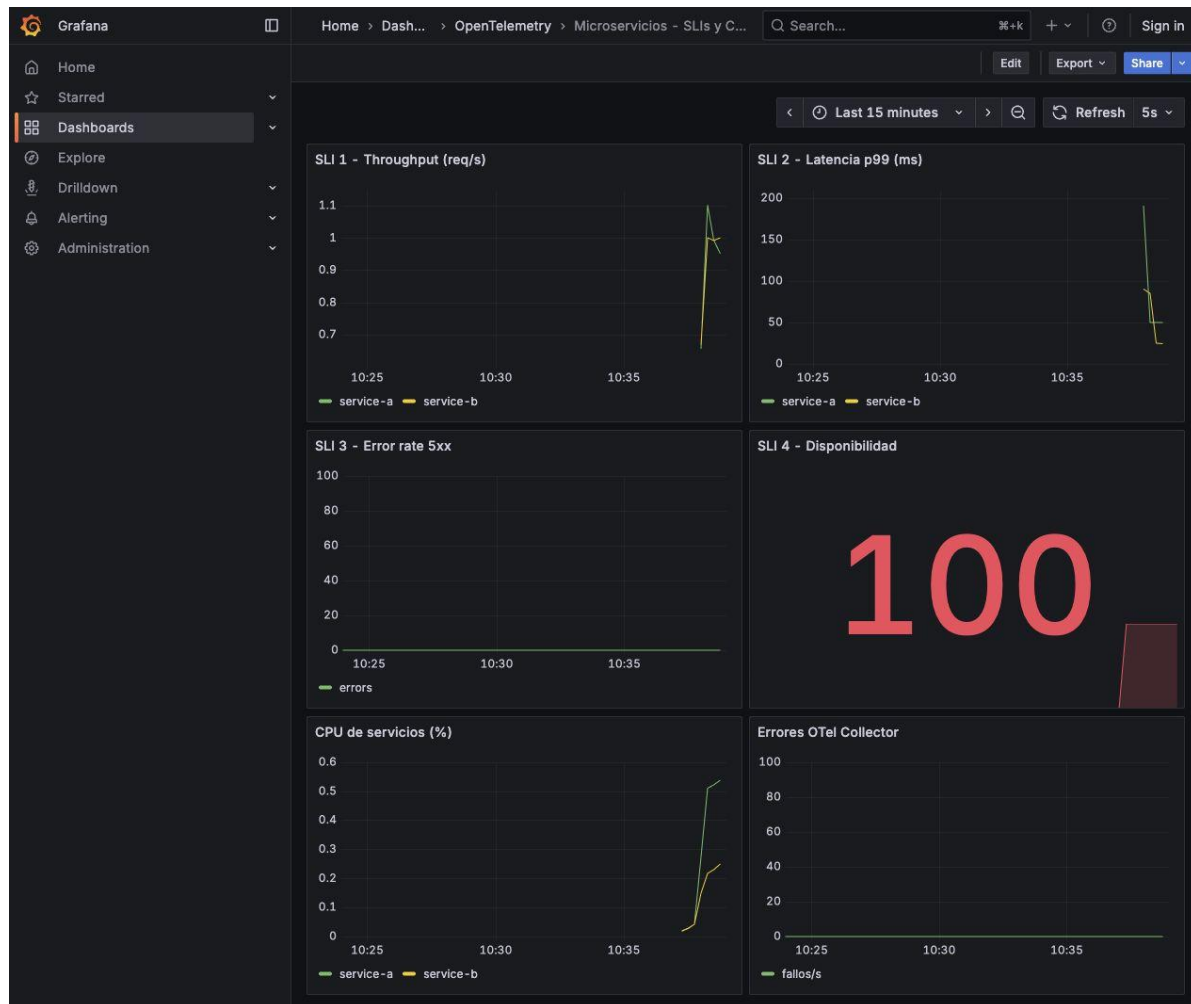


Figura 5. GCP GKE: Grafana con cuatro SLIs, CPU y errores del Collector.

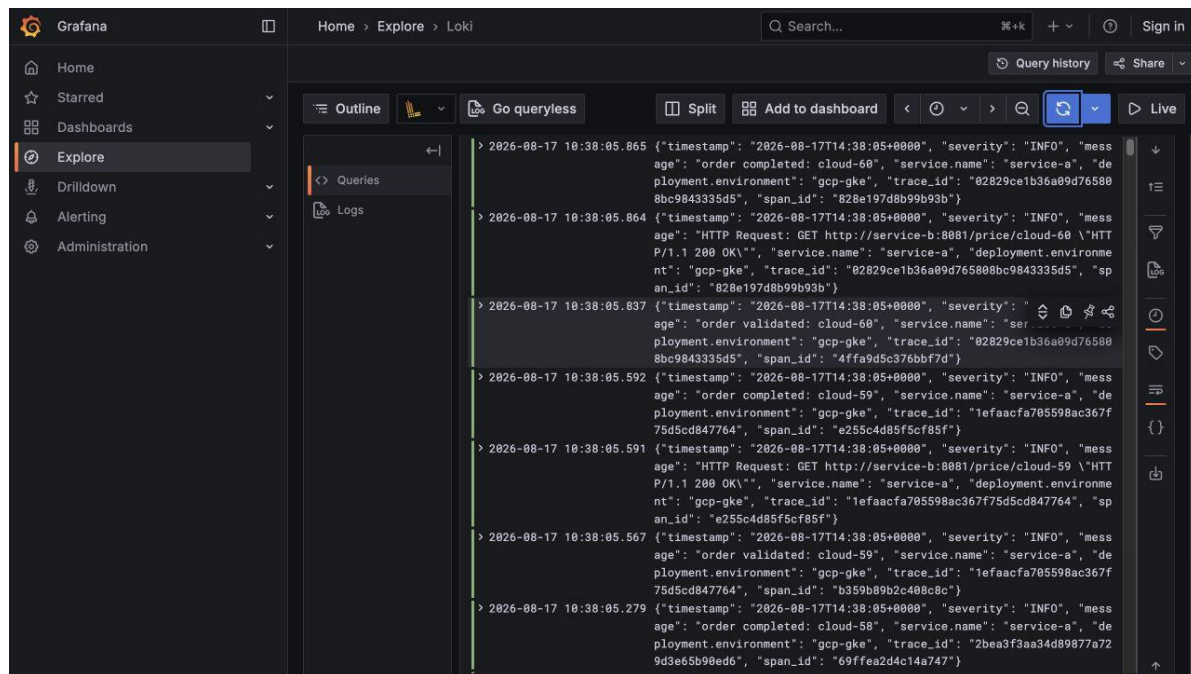


Figura 6. GCP GKE: logs estructurados en Grafana Explore con trace_id y span_id.

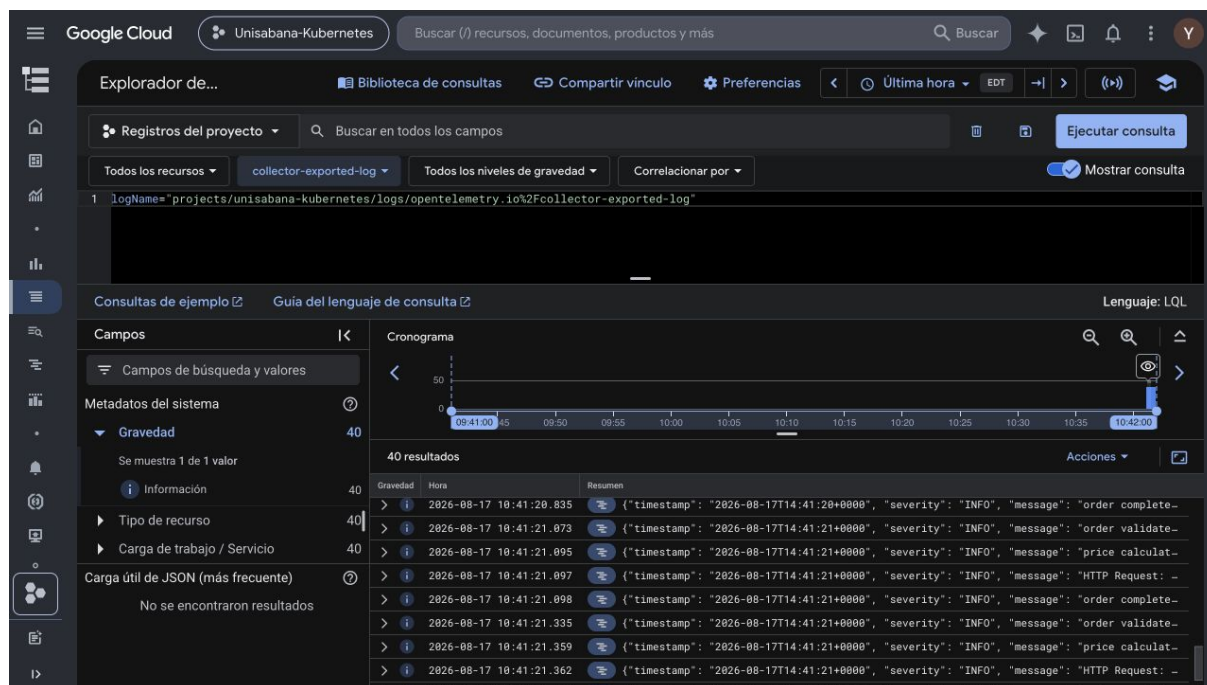


Figura 7. Cloud Logging: registros OTLP de ambos servicios con vínculo nativo a Cloud Trace.

La traza de referencia 1cc2dbbec24fb00661a6f52e206bc38a contiene GET /orders/{order_id}, validate-order, llamada HTTP a service-b, calculate-price y operaciones SQLite CREATE, INSERT y SELECT. Cloud Logging recibió los registros bajo opentelemetry.io/collector-exported-log y conservó el mismo trace_id como campo y como vínculo de traza.

7. AWS ECS Fargate - placeholder de evidencia

Ejecución verificada en un clúster ECS Fargate temporal (cuenta 255191476203, región us-east-1). Trazas de referencia: trace_id e47a885c3d7cfecfe34b8d3e85632cd9 (equivalente en formato X-Ray: 1-e47a885c-3d7cfecfe34b8d3e85632cd9). Detalle completo en AWS-EVIDENCE.md.

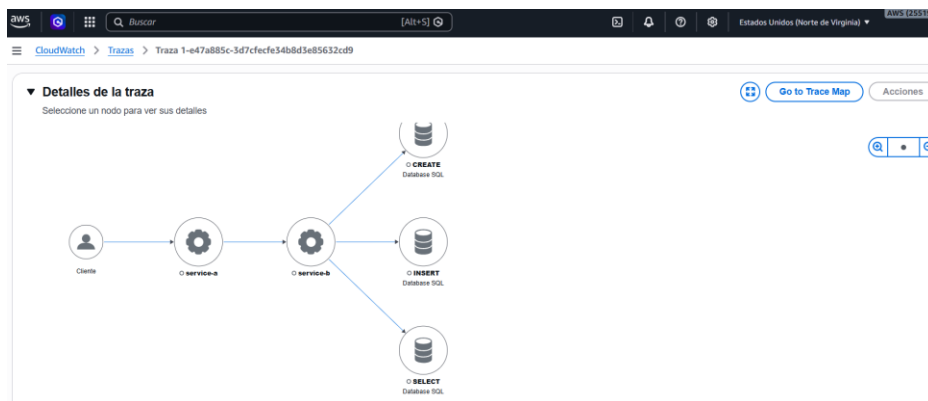


Figura 7. X-Ray: waterfall de la traza distribuida (service-a -> service-b -> CREATE/INSERT/SELECT SQL).

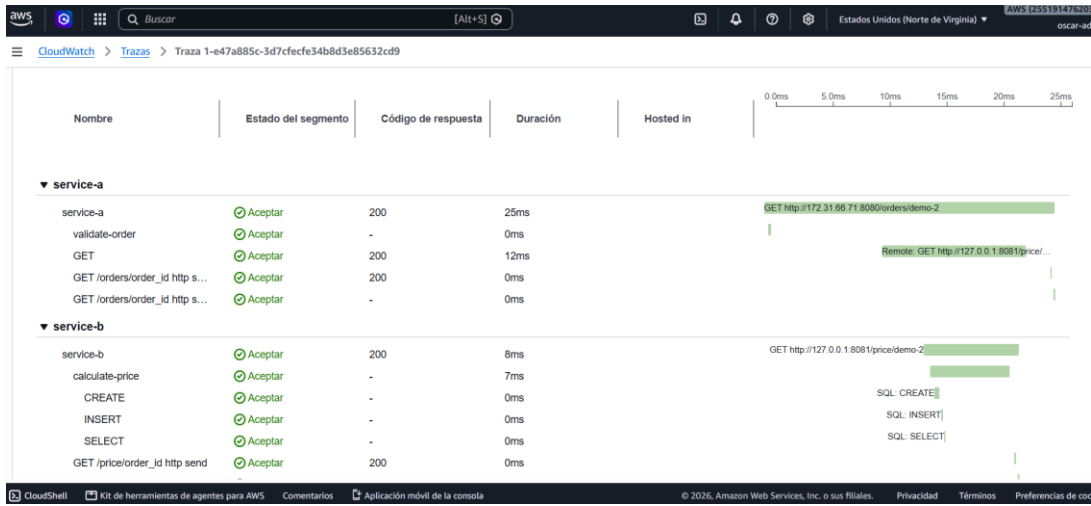


Figura 8. X-Ray: mapa de servicio.

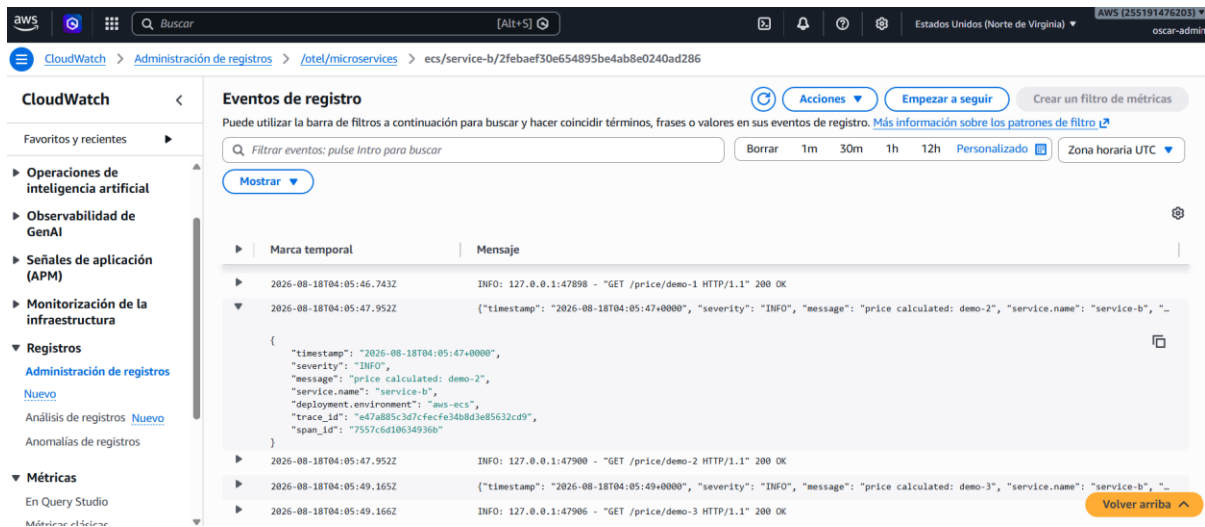


Figura 9. CloudWatch Logs (/otel/microservices, stream ecs/service-b): log JSON con el mismo trace_id que la Figura 7.

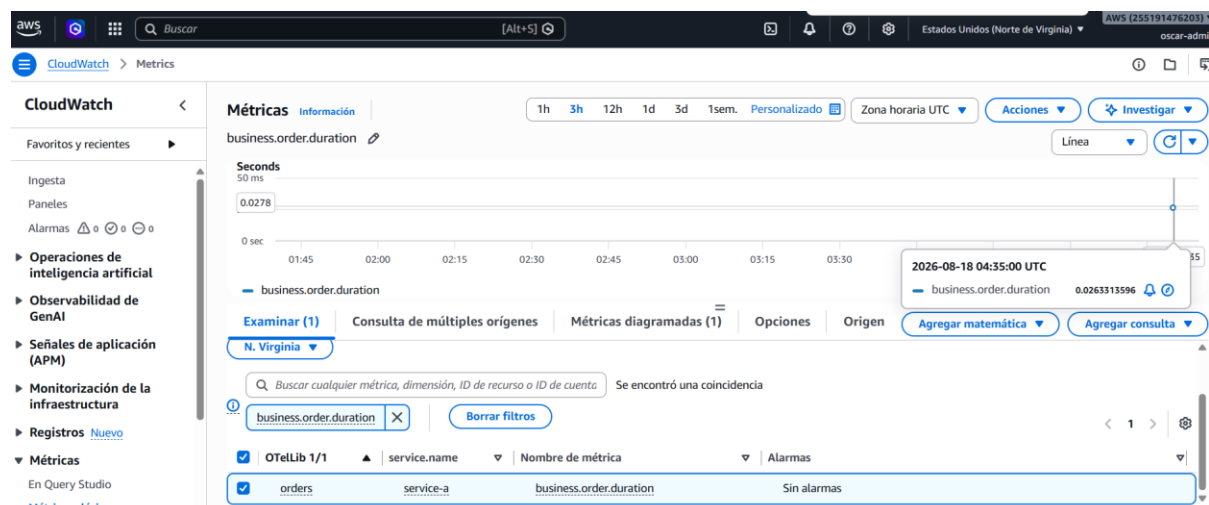


Figura 10. CloudWatch Metrics, namespace ObservabilityLab: business.order.duration con dimensión service.name=service-a, dato real capturado tras el fix del push OTLP.

8. Análisis de overhead

Se ejecutaron dos condiciones con las mismas imágenes, 20 usuarios virtuales y 60 segundos. Baseline usó `OTEL_SDK_DISABLED=true` y la segunda condición habilitó OTel. k6 midió latencia y throughput; docker stats tomó 14 muestras de CPU y memoria conjunta por condición.

Escenario	p99 (ms)	CPU (%)	Memoria (MiB)	req/s
Sin instrumentación	545.23	78.28	150.67	98.61
Con OTel	559.52	88.39	145.55	93.49
Diferencia	+14.29 (+2.62%)	+10.11 (+12.91%)	-5.12 (-3.40%)	-5.12 (-5.19%)

Interpretación

En esta corrida, OTel añadió 14.29 ms al p99 y elevó el uso conjunto de CPU en 12.91%. El throughput bajó 5.19%. La memoria observada fue 5.12 MiB menor con OTel; esta variación debe interpretarse como ruido de asignación, caché y recolección de basura. No hubo errores HTTP en ninguna condición.

Amenazas a la validez

Los resultados corresponden a una sola pareja de corridas en Docker Desktop. Cold starts, caché SQLite, recolección de basura y contención del host afectan la medición. Para una conclusión estadística se recomiendan tres repeticiones alternadas y comparación de medianas.

9. Decisiones de diseño

- Python facilita la lectura académica y dispone de auto-instrumentación madura para FastAPI, HTTPX y SQLite.
- OTLP desacopla las aplicaciones de los backends y permite cambiar destinos en el Collector.
- `memory_limiter` y `batch` controlan presión de memoria y reducen el costo de exportación.
- Jaeger satisface la inspección de trazas; Prometheus y Grafana soportan SLIs; Loki habilita correlación local.
- El rol anónimo Editor de Grafana se usa solo en la demostración local para permitir Logs Drilldown sin login.

10. Reproducción

1. Ejecutar `docker compose up --build -d`.
2. Ejecutar `scripts/smoke-test.sh` para generar tráfico.
3. Validar los cuatro targets en Prometheus.
4. Abrir Jaeger y buscar `service-a`.
5. Abrir el dashboard Microservicios - SLIs y Collector en Grafana.
6. Ejecutar `benchmark/run.sh` y conservar los resultados.

11. Conclusión

El diseño integra los tres pilares y conserva el contexto entre servicios. La combinación de SLIs, trazas y logs correlacionados permite pasar de una alerta a la causa probable. El stack local, el benchmark y GCP GKE fueron ejecutados con evidencia real. AWS ECS permanece como IaC validada y placeholder explícito, pendiente de una ejecución autorizada que no debe sustituirse con capturas simuladas.

Referencias

- OpenTelemetry Python SDK. <https://opentelemetry-python.readthedocs.io/>
- Jaeger Architecture. <https://www.jaegertracing.io/docs/architecture/>
- Grafana Trace Integration. <https://grafana.com/docs/grafana/latest/explore/trace-integration/>
- Grafana k6 Documentation. <https://grafana.com/docs/k6/>
- W3C Trace Context. <https://www.w3.org/TR/trace-context/>